Uncharted Atlas — End-to-End Build Roadmap

From zero to public launch: data, cloud, backend, frontend, APIs, security

Companion to: Uncharted Atlas Day-1 Requirements & Product Spec v1.0 **Purpose:** A concrete, sequential execution plan an engineering team can pick up and follow — every phase names the exact services, accounts, and setup steps, not just the architecture diagram.

How to read this document

The requirements sheet tells you *what* to build. This roadmap tells you *in what order* and *exactly how* — including the boring-but-critical setup steps (account creation, IAM, licensing paperwork) that determine whether Phase 0 takes 4 weeks or 4 months. It's organized as **10 build tracks** running across **4 phases**, matching the phase numbers in Section 12 of the spec, but each track is broken into concrete tickets.

Tracks:

1. Legal & data-licensing groundwork (this is the actual critical path — start it Day 1)
2. Cloud foundation & infra-as-code
3. Data ingestion & storage pipeline
4. Core backend services & API layer
5. Quant/analytics engine (CFA modules)
6. Broker/Account-Aggregator integration
7. Frontend web dashboard
8. Mobile app
9. ML/NLP (sentiment + red-flag detector)
10. Security, observability, CI/CD, and launch

Phase 0 (Weeks 1–4): Foundations

Track 1 — Legal & data licensing (start this the same day as engineering — it's the real bottleneck)

Real-time price data in India cannot legally be shown to end users without a signed data-redistribution agreement, even for a free product. This paperwork has a longer lead time than any code you'll write, so it runs in parallel with everything else from day one.

1. **Incorporate the entity** that will hold vendor contracts (if not already done) — vendors and exchanges contract with a legal entity, not an individual.

2. **Apply for NSE Data & Analytics (NSE DnA) and BSE Market Data redistribution licenses.** These require: entity KYC, a description of the product, expected user volume, and a commercial agreement (fee schedule depends on tick-level vs. delayed data, and on retail redistribution rights). Budget 4–8 weeks for approval.
3. **In parallel, contract a licensed re-distributor** (Truedata, GlobalDataFeeds, or a Kite Connect market-data feed) as a faster path to real-time data while the direct exchange license is pending — many products launch on a re-distributor's licensed feed first, then move to a direct exchange agreement at scale.
4. **Fundamentals/financials vendor**: shortlist Tickertape API, Trendlyne, or Refinitiv Eikon/Datastream (Section 4.1 of the spec) — get sandbox/trial API keys immediately so backend engineers can build against real schemas from week 1, and run the commercial negotiation in parallel.
5. **News vendor**: PTI or Reuters News API commercial terms, or an aggregated licensing deal with Indian financial media. Get a trial feed key immediately.
6. **Confirm the "Traction" ambiguity** flagged in the spec — before contracting anything under that name, verify with the original stakeholder whether they meant Refinitiv, Tickertape, TrendlyneAPI, Truedata, or something else. Don't let an unverified vendor name block the shortlist above.
7. **SEBI registration question**: engage compliance counsel now to determine whether any planned feature (screener flags, red-flag detector, factor tilts) crosses from "analytics/education" into "Research Analyst/Investment Adviser" territory under SEBI rules. This determines what disclaimers and what feature gating are needed before Phase 3 launch — resolve it early, not at the end.
8. **DPDP Act 2023 review**: data-privacy counsel drafts the consent flows, privacy policy, and data-retention policy for PII and broker tokens before the token vault (Track 6) is built, since the vault's design depends on the legal retention requirements.
9. **Broker API agreements**: register as a developer/partner with each broker you plan to support (Zerodha Kite Connect, Upstox API, Angel One SmartAPI, ICICI Direct Breeze, Fyers API, 5paisa, Dhan API). Each has its own developer console, app-registration process, and (for some) a personal API-token fee model — get sandbox credentials for at least 2–3 brokers in week 1.
10. **RBI Account Aggregator (AA) ecosystem**: apply to onboard as a Financial Information User (FIU) with the Sahamati/AA framework, and open technical discussions with at least one licensed AA (Perfios, Finvu, OneMoney). This is a separate, slower-moving compliance track — start it in Phase 0 even though it's only *used* starting Phase 2.

Track 2 — Cloud foundation & infra-as-code

1. **Choose the cloud provider and region.** AWS or GCP, primary region in Mumbai (`ap-south-1` on AWS, `asia-south1` on GCP) for latency to NSE/BSE colocated servers; secondary region (e.g., Hyderabad-adjacent or a second AWS AZ set) for DR.

2. **Set up the organization/billing structure**: root/management account, separate sub-accounts (or projects) per environment — `dev`, `staging`, `prod` — using AWS Organizations or GCP folders/projects, so blast radius and IAM are isolated per environment from day one.
3. **Stand up Terraform** as the infra-as-code baseline:
   - Create a private Git repo (`infra/`) with a remote state backend (S3 + DynamoDB lock table on AWS, or GCS + Cloud Storage locking on GCP).
   - Write the first Terraform modules: VPC (multi-AZ, public/private subnets), IAM roles/policies, EKS/GKE cluster, and a bastion/VPN or SSM-based access path (no direct SSH to prod).
4. **Provision Kubernetes (EKS or GKE)**, multi-AZ, with separate node pools for: general services, the streaming/low-latency tier (Go/Rust services — consider dedicated, higher-network-throughput instance types), and the ML/analytics tier (larger memory instances for Python/quant workloads).
5. **Install the service mesh** (Istio or Linkerd) for mTLS between services and built-in observability hooks — do this before services proliferate, not after.
6. **Set up Terraform Cloud/Atlantis** (or GitHub Actions-driven `terraform plan/apply`) so infra changes are peer-reviewed and applied consistently, not applied by hand from a laptop.
7. **DNS & domain**: register `unchartedatlas.com` and `.in` (Section 11's single-brand rule), point to a managed DNS zone (Route 53 / Cloud DNS), and provision TLS via ACM or Let's Encrypt/cert-manager on the cluster ingress.
8. **CDN & static hosting**: CloudFront or Cloud CDN in front of the Next.js frontend and static assets.

Track 3 — Repositories, environments, and team scaffolding

1. Create the repo structure: `infra/` (Terraform), `services/` (one repo-per-service or a monorepo with `services/<name>` — a monorepo with clear service boundaries is usually easier at this team size), `frontend-web/`, `frontend-mobile/`, `analytics-engine/`, `docs/`.
2. Stand up GitHub (or GitLab) org, branch-protection rules, and required-review policy on `main`.
3. Bootstrap CI: GitHub Actions workflows for lint/test/build on every PR, with SAST (Semgrep), dependency scanning (Snyk), secrets scanning (gitleaks/TruffleHog), and container scanning (Trivy) wired in from the first merged PR — not retrofitted later.
4. Set up Slack/Linear/Jira for the team and connect CI status + on-call alerting.
5. Hire/onboard against the Section 10 team roster; the security engineer and DevOps/SRE should be among the first hires since Tracks 2, 3, and 10 depend on them.

Phase 1 (Weeks 5–12): Core pricing + risk-analytics MVP

Goal of this phase: a working six-pane dashboard showing live prices and the Section 5.1 portfolio/risk-analytics suite, backed by a single broker connection. This is the smallest slice that proves the core USP (free, real-time, CFA-grade analytics).

Track 3 — Data ingestion & storage pipeline

1. **Provision the streaming backbone**: deploy Kafka (or Redpanda, which is often simpler to operate) on Kubernetes via a managed operator (Strimzi for Kafka), or use a managed service (MSK on AWS, Confluent Cloud) to avoid operating Kafka yourselves in the early weeks.
2. **Build the market-data ingestion service** (Go or Rust): connects to the licensed feed (Truedata/GlobalDataFeeds/Kite Connect market feed chosen in Track 1), normalizes ticks into a common internal schema (instrument ID, price, volume, timestamp, exchange), and publishes to Kafka topics partitioned by instrument.
3. **Provision time-series storage**: ClickHouse or TimescaleDB (TimescaleDB is Postgres-based and often faster to get a team productive on if the team already knows SQL/Postgres; ClickHouse wins on raw tick-ingest throughput at scale). Stand up a managed instance or a Kubernetes-operator-managed cluster, and build the Kafka-to-TimeSeries-DB sink consumer.
4. **Provision the operational database**: PostgreSQL (multi-AZ, e.g., RDS or Cloud SQL) for user accounts, portfolios, watchlists, and broker-token *metadata* (never raw tokens — see Track 6).
5. **Provision Redis** (or KeyDB) for session state, computed-metric caching, and pub/sub fan-out — this is what lets thousands of WebSocket clients receive the same recalculated Sharpe ratio without hitting Postgres or the quant engine on every tick.
6. **Provision Elasticsearch/OpenSearch** for instrument search (ticker/company name autocomplete) — needed for the command palette (Section 6) even in the MVP.
7. **Build the streaming fan-out gateway**: a Go/Rust WebSocket (or gRPC-streaming) service that subscribes to Redis pub/sub / Kafka and pushes live price + recalculated-metric updates to connected frontend clients.

Track 4 — Core backend services & API layer

1. **User/auth service** (Node.js/NestJS or Go): email/OTP or OAuth-based signup/login, JWT session issuance, integrates with Postgres for the user table. Add rate limiting and standard OWASP protections from the start.
2. **API gateway**: stand up Apollo (Node) or Hasura (auto-generated GraphQL over Postgres) as the GraphQL layer for client queries (watchlists, portfolio CRUD, layout preferences), plus a dedicated gRPC/WebSocket channel (built in Track 3.7) for streaming ticks/analytics — these are two separate transport paths by design, don't try to force streaming through GraphQL subscriptions at scale.

3. **Portfolio ledger service**: stores user holdings (manually entered in MVP, broker-synced from Phase 1 end), computes position-level P&L, and is the source of truth the quant engine reads from.
4. **BFF (backend-for-frontend)**: a thin service or GraphQL resolver layer that composes calls to the above services into the shapes the dashboard widgets need, so the frontend isn't making 6 separate calls per pane.

Track 5 — Quant/analytics engine (Section 5.1 first)

1. **Stand up the Python/FastAPI analytics service**, with NumPy, SciPy, statsmodels, and QuantLib as core dependencies.
2. **Build the rolling-computation engine**: on every new price tick relevant to a user's holdings (consumed from Kafka), recompute per-holding and portfolio-level Sharpe, Sortino, Treynor, M^2, Jensen's alpha, and Information Ratio, using a benchmark series (Nifty 50/Sensex, pulled from the same market-data pipeline).
3. **Cache computed metrics in Redis** keyed by `user_id:portfolio_id:metric`, with the WebSocket gateway (Track 3.7) pushing changed values to connected clients — this avoids recomputing on every single client request.
4. **Build the correlation/covariance matrix service**: computes the live heat-map across a user's holdings and major indices — this feeds the "showcase" widget in the six-pane dashboard.
5. **Version every calculation function** (formula + parameters) so results are reproducible — this is what makes the Section 5.7 "algorithm transparency" pages possible later; don't defer this versioning to Phase 3, bake it into the engine's design now.

Track 6 — Broker integration (single broker for MVP)

1. Pick the first broker to integrate (Zerodha Kite Connect is a common first choice given documentation maturity).
2. **Build the broker-integration gateway service**: implements the OAuth redirect flow — user clicks "Connect Zerodha," is redirected to Zerodha's own login screen, authorizes, and Zerodha redirects back with an auth code that the gateway exchanges for a scoped access token. The user's broker username/password is never seen or stored by Uncharted Atlas.
3. **Build the token vault**: a dedicated service backed by AWS KMS/CloudHSM (or GCP Cloud KMS) for envelope encryption of tokens at rest in Postgres; implement automatic token refresh/expiry handling per Zerodha's API terms.
4. **Build the portfolio-sync job**: on connection (and periodically thereafter), pulls holdings/transactions from the broker API and writes them into the Portfolio ledger service (Track 4.3), which the quant engine (Track 5) then computes against.
5. **Build the "Connected Accounts" settings page backend**: an endpoint to list and revoke connections — required for both trust and DPDP-adjacent expectations (Section 7).

Track 7 — Frontend web dashboard (MVP: 2–3 of the six panes)

1. **Scaffold the Next.js (App Router, TypeScript) project**, set up the design system (Section 6's dark, terminal-inspired theme with a light-mode toggle) using Tailwind or CSS variables, per the frontend-design principles of intentional, non-templated visual choices.
2. **Set up TanStack Query** for REST/GraphQL data fetching/caching, and **Zustand or Redux Toolkit** for client-side UI state (pane layout, selected tickers).
3. **Build the WebSocket client** connecting to the streaming gateway (Track 3.7) for live price and metric updates; wire it into TanStack Query's cache so panes re-render on tick.
4. **Build the six-pane workspace shell**: a drag-to-resize, drag-to-rearrange grid (e.g., using `react-grid-layout`) with a widget registry pattern so each pane type (watchlist, chart, risk scorecard, etc.) is a pluggable component. Persist layouts per user via the BFF.
5. **Build the first three widgets for MVP**: live watchlist, price chart (TradingView Lightweight Charting Library), and the portfolio risk-metric scorecard (Sharpe/Sortino/M^2 from Track 5).
6. **Build the command palette** (type a ticker, jump to it) using the Elasticsearch-backed instrument search (Track 3.6).
7. **Build the broker-connect screen and Connected Accounts settings page**, calling the Track 6 backend.

Phase 2 (Weeks 13–20): Fixed Income, Derivatives, FSA, multi-broker, news

Track 5 (continued) — Remaining quant modules

1. **Fixed Income module**: build the duration/convexity/yield-curve calculators (Section 5.3) in the quant engine, sourcing G-Sec and corporate bond price data from the fundamentals/market-data vendor; add the credit-spread and callable/putable adjustment logic.
2. **Derivatives module**: build the options Greeks engine (delta/gamma/theta/vega/rho) recalculated on every tick, the multi-leg payoff-diagram builder, put-call parity checker, and implied-volatility surface — this is computationally heavier, so benchmark it early and consider a dedicated node pool or even a Rust implementation for the hot path if Python proves too slow for tick-rate recalculation.
3. **Financial Statement Analysis module**: build the full ratio suite, 3-step/5-step DuPont decomposition with historical trend charts, and the common-size statement viewer, sourced from the fundamentals vendor's financial-statement data.

Track 6 (continued) — Multi-broker + Account Aggregator

1. Add OAuth integrations for the remaining brokers (Upstox, Angel One, ICICI Breeze, Fyers, 5paisa, Dhan) using the same token-vault pattern established in Phase 1 — each broker's OAuth flow differs slightly, so budget 1–2 weeks per additional broker for integration + testing.
2. **Build the unified-portfolio aggregation logic**: merge holdings across multiple connected brokers into a single portfolio view, with the correlation matrix and risk scores now computed on the combined position set.
3. **Integrate the RBI Account Aggregator flow**: once your FIU onboarding (Track 1.10) is approved, integrate with your chosen AA partner (Perfios/Finvu/OneMoney) as the compliant "one-login" path for users who prefer consent-based aggregation over direct per-broker OAuth.

Track 9 — News + sentiment engine (new track starts here)

1. **Stand up the news ingestion service**: consumes the licensed news feed (PTI/Reuters/aggregated RSS) and publishes normalized articles to Kafka.
2. **Build the sentiment-tagging model**: fine-tune or prompt a transformer model (HuggingFace, e.g., FinBERT-style) to tag each headline positive/neutral/negative.
3. **Build the news-to-holding matcher**: use a vector database (pgvector, Pinecone, or Weaviate) to embed article text and match it semantically to portfolio holdings/tickers, so news auto-links to the relevant position.
4. **Build the unified news feed widget** on the frontend, with per-headline sentiment tags, and wire it into the six-pane widget registry as a new pane option.

Track 7 (continued) — Remaining dashboard panes

Add the correlation heat-map, options Greeks panel, sector heat-map, news+sentiment feed, and DuPont/ratio-breakdown panes to the widget registry, plus the shareable layout templates ("Options Trader," "Long-Term FI Investor," "Index Tracker") mentioned in Section 6.

Track 8 — Mobile app

1. Scaffold React Native, sharing the design tokens and a subset of the analytics SDK/components with the web app where feasible.
2. Prioritize: watchlist, portfolio scorecard, and a simplified 1–2 pane view for mobile — the full six-pane workspace is a desktop-first pattern, so mobile should have its own simplified information architecture rather than a cramped port of the desktop grid.

Phase 3 (Weeks 21–28): Economics, ML red-flag detector, SEO, security audit, launch

Track 5 (final module)

Economics module: macro dashboard (GDP growth, CPI/WPI inflation, repo rate, business-cycle phase indicator, INR exchange rate) and an economic calendar with market-impact tagging (RBI policy, Union Budget, US Fed decisions) — sourced from the macro/global-data vendor selected in Track 1.

Track 9 (continued) — ML red-flag detector + factor screener

1. **Build the financial red-flag detector**: an anomaly-detection model (trained on historical financial-statement data) that flags aggressive revenue recognition, deteriorating cash conversion, and rising related-party transactions — treat this as a genuine ML project with its own train/validate/backtest cycle, not a bolt-on script, since it's called out as a strong USP and will be scrutinized by users.
2. **Build the factor/smart-beta screener**: value/growth/quality/momentum/size factor exposure and screening, integrated with the portfolio-level active-share calculation from Track 5's Phase-1 work.

Track 10 — Security, observability, compliance sign-off, and launch prep

1. **Observability**: deploy Grafana + Prometheus + Loki/Tempo (or Datadog) across all services; build dashboards for latency (sub-second tick-to-UI target), error rates, and Kafka consumer lag; wire alerting to on-call.
2. **Full third-party penetration test** of the token vault, auth flows, and public API surface — schedule this with enough lead time (typically 2–4 weeks including remediation) before the target launch date.
3. **SOC 2-equivalent control review**: formalize access-control, change-management, and incident-response policies even if a full SOC 2 audit is deferred, since it's referenced as a non-functional requirement.
4. **Compliance sign-off**: confirm with legal/compliance counsel that no shipped feature (screener flags, red-flag detector) has drifted into SEBI Research Analyst/Investment Adviser territory without the corresponding registration; finalize DPDP-compliant privacy policy and consent flows; confirm NSE/BSE data-redistribution license is fully executed (not just in progress) before public launch.
5. **SEO build-out (Section 11)**: implement the meta-title pattern and per-module descriptions; build the content-rich educational landing pages ("What is the Sharpe Ratio," "M-squared Explained," etc.) with schema.org `FinancialProduct`/`Article` structured data; run a Core Web Vitals audit (target sub-2.5s LCP) on the Next.js app, optimizing image loading, code-splitting, and CDN caching as needed.
6. **Load and scale testing**: simulate national-scale concurrent-usage spikes (market open, budget day, results season) against the streaming gateway and quant engine, and tune Kubernetes autoscaling (HPA/cluster autoscaler) and Kafka partition counts accordingly.

7. **Algorithm-transparency pages**: publish the model/methodology pages for each score (Sharpe, M^2, red-flag detector, etc.), drawing on the calculation-versioning work done in Track 5 back in Phase 1 — this is far easier if that versioning was built in from the start rather than retrofitted here.
8. **Launch**: public release, with the free-core-tier / credit-based-premium-tier split (Section 7) live, and monitoring/on-call actively staffed for the first weeks post-launch.

Cross-cutting notes worth flagging to the CTO

- **The legal/licensing track (Track 1) is the true critical path**, not the engineering build. NSE/BSE data agreements, SEBI compliance review, and RBI AA onboarding all have external approval timelines the engineering team can't accelerate — starting these in week 1, in parallel with infra setup, is what keeps Phase 3's public-launch date realistic.
- **Build the calculation-versioning and audit-trail pattern (Track 5.5) early**, not in Phase 3. Retrofitting reproducibility into an analytics engine that's already computing six modules' worth of metrics is significantly more expensive than designing for it from the first Sharpe-ratio calculation.
- **Keep the credential-security boundary strict from day one**: the token vault and OAuth-only broker connection pattern (Track 6) is both a compliance requirement and the platform's biggest security liability if done wrong — treat it as security-engineer-owned, code-reviewed by the security hire specifically, not just another backend service.
- **The "Traction" vendor name needs resolving before any contract is signed** — confirm with the original stakeholder which vendor was actually meant, using the shortlist in Track 1.4 as the working set of realistic candidates.